ENGINEERING REFERENCE · VERSION 1.0

SmartHub / Anton

Technical Documentation

Automated bid-recommendation system for insurance lead-pings — purpose, assumptions & context, high-level design, low-level design, APIs, and known limitations.

August 19, 2026

Contents

Automated bid-recommendation system for insurance lead-pings.

Version 1.0 · Living document · Engineering reference

This document describes the SmartHub / Anton system end to end: what it is, the business and modeling assumptions it rests on, its high-level architecture (HLD), the low-level design of each component (LLD), the serving APIs, and the current known limitations and roadmap. It is written for engineers and technical stakeholders who need to build on, operate, or review the system.

---

1. Introduction & Purpose

1.1 What SmartHub and Anton are

SmartHub is a **lead marketplace / reseller** for insurance leads. It does not generate leads itself; it sits in the middle of a two-sided market. **Upstream partners (producers)** have leads and offer them to SmartHub in real time; **downstream buyers (agents)** purchase those leads from SmartHub. SmartHub buys from partners and resells to buyers, keeping the spread.

Anton is the data-science bidding engine inside SmartHub. Its single responsibility is to answer, for every incoming lead offer (a "ping"): **what price should we bid?** Anton must do this programmatically, profitably, and *explainably* — the business requirement is an auditable bidder, not a black box.

1.2 The problem Anton solves

The bid decision is framed as a **profit-maximisation** problem, not a raw prediction. For a given lead, the model estimates the **probability of winning as a function of the bid** (higher bids win more often). The system then selects the bid that maximises **expected profit**:

```
expected_profit(bid) ≈ P(win | bid) × (expected_revenue − bid)
```

subject to two bounds: an upper bound (the **ceiling**) set by expected revenue and a target **contribution margin (CM)**, and a lower bound (the **floor**) set by any partner-side minimum bid. Win-probability is required to be **monotonically non-decreasing in bid**, enforced inside the model, so the economics can never invert.

Everything is organised **per lead type** (`auto = 6` , `home = 1`). Each lead type has its own training data, model, evaluation, and serving pointer. Adding a new type (e.g. commercial) is designed to be a registry entry, not a code fork.

1.3 Goals and non-goals

Goals. Choose the profit-maximising bid per ping; be at least as good as the current production bidder on profit and CM; remain transparent and maintainable; serve the bid decision within a **≤ 1 second turnaround (TAT)** at the target throughput; log every decision with the exact model, features, and config that produced it.

Non-goals (current phase). Anton does not place bids in production yet — staging is **predict-only** and fire-and-forget. It does not model buyer reject rates (already baked into "expected revenue"), does not handle buyer-pacing (`bpfm_score` , a backend concern), and does not decide downstream buyer routing.

1.4 Design philosophy

The system prioritises **auditability over cleverness**. Every bid follows one of a small number of explicit, logged decision paths; data validation **reports** rather than silently mutating data; and every prediction is persisted with full lineage. This philosophy is the lens through which the design choices below should be read.

---

2. Business Context

2.1 The marketplace and its players

SmartHub is the intermediary between producers and agents.

Partners (upstream / sellers). Companies that send leads (~6 total, ~4 active — e.g. Healthy Labs, Launch Potato, Malva). In the portal they are called **accounts**. A partner has one or more **campaigns** (`campaign_id`); most have one, and Malva is the only one with two (a home and an auto campaign). Volume is intentionally conservative today and expected to grow as larger partners onboard, so low per-segment sample counts now are expected to improve.

Buyers (downstream / agents). The agents SmartHub resells to, across marketplaces such as Insurance.io (~200 accounts), SF Pro, and BPIO (1,000+ accounts). Different buyers purchase different products (leads / calls / clicks), mostly only one of the three.

2.2 How one lead flows

1. **Ping.** A partner sends a ping — "here's a lead, what will you pay?" — with non-PII attributes (state, city, vehicles, age, accident history, currently-insured flag, etc.).
2. **Buyer match.** SmartHub instantly checks its buyer distribution: which active buyer campaigns match this lead and what they'd pay. The backend ranks everyone into **listings**.
3. **Bid.** SmartHub bids back to the partner (this is Anton's decision).
4. **Win.** If the partner accepts the bid, SmartHub has **won the lead** and then tries to resell it downstream.

A representative funnel from the domain walkthrough: ~19,900 pings → ~76% returned a valid bid → ~5,698 leads won → roughly half successfully resold.

2.3 Money mechanics

The **reseller rule**: SmartHub only owes the partner money if it actually resells the lead to at least one buyer. Winning a lead whose buyers all reject is a harmless "dead ping" (no cost, no revenue). A **shared** lead where one buyer accepts and one rejects can still leave SmartHub paying the partner while collecting less — hurting contribution margin. Early on, over-aggressive bidding lost money outright (day one lost ~$2,600), which is why bidding was tuned down and why a disciplined, learnable bidder matters.

Expected revenue is the key input for Anton. A separate signal stores what SmartHub expects to make from a lead, **already discounted by how often buyers reject**. For example, a top buyer paying $78 who accepts only 20% of the time contributes ~$15.90 of expected revenue. This is why revenue can look much larger than the bid and why margins look fat — the reject rate is already baked in, so Anton does not model it separately.

Contribution margin (CM) is the lever. The legacy `dumb 10 / 25 / 50 / 75` strategies are simply CM targets: at 25% CM, a lead worth $10 is bid at $7.50. A deliberate ±10% variance was applied to each bid to sample many price points and learn the win-rate curve.

2.4 "Payout" is overloaded — concept-to-column map

The warehouse uses "payout" on the **buyer (downstream) side**, the opposite of a reseller's usual usage. The table below is the authoritative mapping.

Concept	Table.column
Our bid to the partner (our cost)	`lead_pings.bid`
Realized revenue	`lead_pings.rev` (≈ Σ `lead_ping_listings.payout` over accepted)
Expected revenue (ceiling input)	`lead_pings.exp_rev` (backend field) — *interim:* Σ `lead_ping_listings.est_payout` per ping
Won the lead (partner accepted)	`lead_pings.won`
Downstream accept / reject	`lead_ping_listings.post_accepted`
Listing selected / excluded / deduped	`lead_ping_listings.selected` / `excluded` / `de_duped`
CM-target lever (dumb 10/25/50/75)	`lead_pings.bidding_strategy_id`
Exclusive vs shared lead	`lead_ping_listings.exclusive`

On `lead_pings`, `bid` is what SmartHub pays the partner (cost) and `rev` is realized revenue. On `lead_ping_listings`, `payout` / `est_payout` are **buyer-side** money flowing *to* SmartHub — not what it pays the partner.

---

3. Assumptions & Constraints

This section makes explicit the assumptions the system rests on. Several carry a dated status because they were resolved (or remain open) across DS meetings.

3.1 Modeling assumptions

- **Objective.** Predict win rate, then optimise expected profit. Find the ceiling that holds the CM target, but **bid lower when win rate is unchanged** (capture the "shelf/edge"). This is a single optimisation, not an either/or of "maximise win rate" vs "maximise CM".
- **Monotonicity.** Win-probability is assumed and enforced to be monotonically non-decreasing in `bid` (a monotone constraint on the tree models; isotonic calibration preserves it). This makes the profit curve well-behaved.
- **Bounds may not exist.** The floor/ceiling and the win-rate curve "may or may not exist for a given partner / lead type." Discovering *whether* and *where* bounds exist is part of Anton's job, not a given.
- **Exploration is required.** Anton must bid with deliberate variability around its own optimum (explore/exploit) to learn the market's shape at other price points, not merely exploit the current best estimate.
- **Recency.** The market shifts, so old data goes stale. "Recent" is an explicit, configurable rolling window (default 21 days for the training table), never buried in code.
- **Cold start.** When there is no recent data (new partner/lead type), behaviour must be an explicit, articulated fallback rather than chaotic.

3.2 Data assumptions

- **Label logic (7 Jul 2026).** A partner ping is recorded; if it passes validation a bid is recorded; on **error the record ends** (no auction). A placed bid with `won` null is a **loss**; a win sets `accepted=true`. For training: **drop `erred` rows**, treat a placed bid with null `won` as a loss, and keep all non-errored pings (they carry competitor-pricing signal).
- **Expected revenue source.** The intended source is the backend field `lead_pings.exp_rev`. As of 7 Jul 2026 it began populating; the pipeline coalesces `exp_rev` (> 0) with an interim listings-sum fallback (Σ `est_payout` per ping). Earlier the field was entirely unpopulated, which is why the listings-sum stopgap exists.
- **Missing data is signal, minimally.** Providers send default values regardless of the real consumer (e.g. `marital_status` often defaults to "single"). So the system does **not** mean-impute: string features get a literal `"NAvail"`, `age` gets a `-1` sentinel plus an `age_missing` flag. No per-ping completeness/reliability features are built — `traffic_tier` already carries partner-subsource quality into the model.
- **Time zone is Pacific.** The operational day runs to ~5:30pm PT, so time features (`created_hour`, `created_dayofweek`, `is_workday`) are Pacific, and observed holidays are marked explicitly (`config/holidays.json`).
- **`current_carrier` is excluded** from the model for now (sometimes populated even when `insured=false` — bad data), though it is critical for the business rule of not reselling to a lead's current carrier.

3.3 Operational assumptions

- **Single node.** The entire stack runs on one host via `docker-compose`.
- **Serving is an API ping, not a DB lookup.** At serve time Anton receives the lead as a full payload over HTTP, pared to the needed features.
- **Staging is decoupled and predict-only.** Async, fire-and-forget; no bids are placed in production yet.
- **Latency / throughput SLA.** `/recommend_bid` must serve **200 requests/minute at ≤ 1 s turnaround** per request; the design was validated to that target (and 6× headroom).
- **Baseline to beat.** The current production Anton runs with no losses but lower profit. The new model must be **≥ the current system** on profit/CM; the MVP target was end of Q3.

3.4 Open questions / to confirm

Confirming `bid_to_use`; the precise distinction between `accepted` (resold to ≥1 buyer), `won` (partner accepted our bid), and `accepted_listings`; and full population/coverage of `exp_rev` across lead types before switching off the interim listings-sum fallback.

---

4. High-Level Design (HLD)

4.1 System at a glance

The system is a **four-stage offline batch pipeline** that produces a promoted model, plus an **online serving path** that uses it. The whole thing runs on a single node, orchestrated by Prefect, with GitHub Actions building images that watchtower auto-deploys.

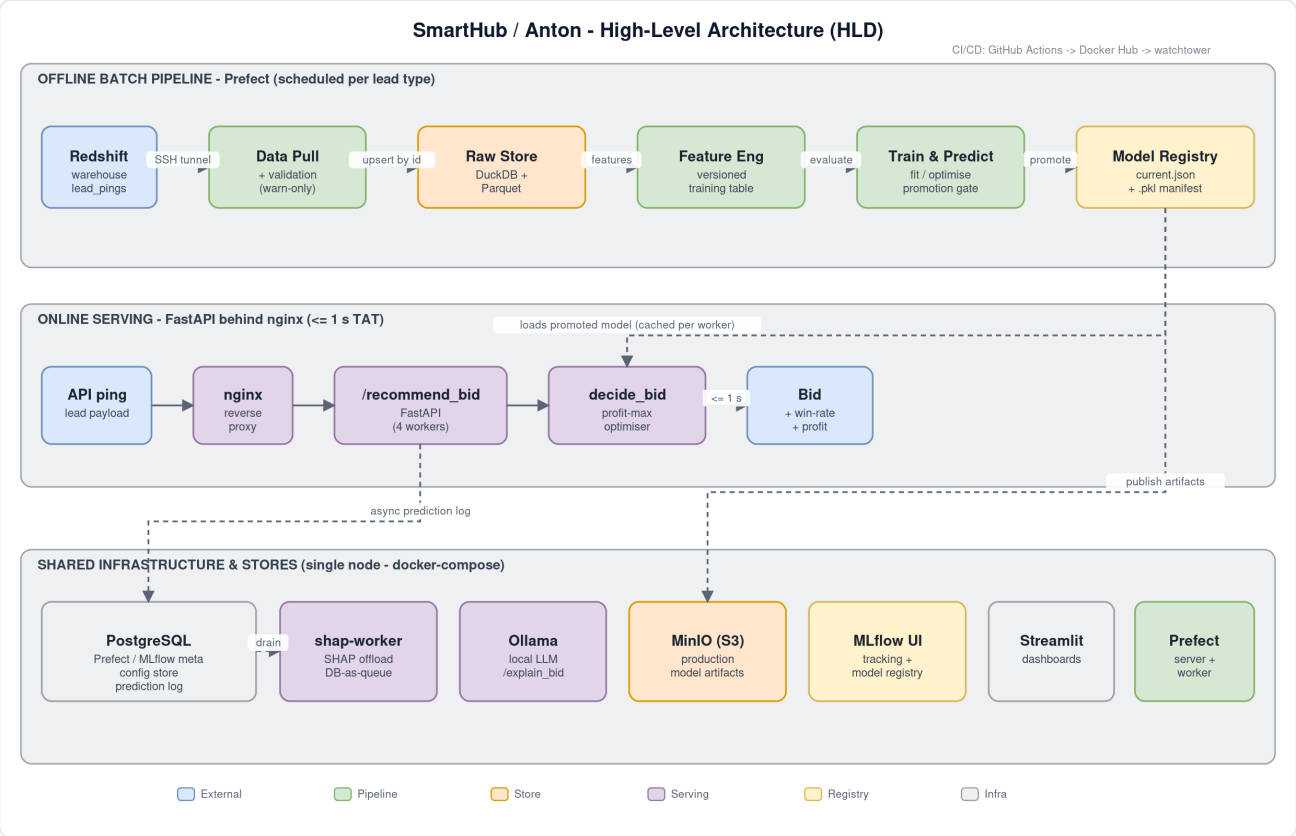

Figure 1 — High-level architecture. The offline Prefect pipeline (top) produces a promoted model in the registry; the online FastAPI serving path (middle) loads that model to answer `/recommend_bid` ; shared infrastructure and stores (bottom) back both. Dashed arrows are asynchronous / off the bid path.

4.2 The four stages and the serving path

1. **Data Pull (STEP 1).** Pull `lead_pings` (and joined expected-revenue) from Redshift over an SSH tunnel, incrementally, into a raw store (DuckDB + Parquet). Validate each batch (warn-only).
2. **Feature Engineering (STEP 2).** Turn the raw store into an immutable, versioned, leakage-safe **training table** driven by a single feature registry.
3. **Train & Predict (STEP 3).** Fit a calibrated, monotone win-probability model, run an offline bid-optimiser evaluation, and decide promotion against the currently-serving model.
4. **Model Registry.** Persist versioned artifacts (file registry + MLflow + S3/MinIO) and flip the serving pointer atomically on promotion.
5. **Serving (online).** A FastAPI app loads the promoted model and answers `/recommend_bid` on the hot path, logging every decision asynchronously and offloading SHAP explanation to a separate worker.

4.3 Component / service topology

The single-node `docker-compose` stack is composed of the following services.

Service (container)	Image basis	Role
`nginx`	nginx:alpine	Only host-exposed entry point; reverse-proxy to the API.
`serve` (`smarthub-serve`)	serve image	FastAPI bid API; `SERVE_WORKERS` (default 4) preforked uvicorn processes.
`shap-worker`	serve image	Drains the prediction log and backfills SHAP explanations off the bid path.
`slo-alerts`	serve image	Periodic SLO checker / alerter (60 s loop).
`worker` (`prefect-worker`)	worker image	Executes the Prefect pipeline flows (pull → features → train).
`prefect-server`	prefecthq/prefect:3	Orchestration API / UI and scheduler.
`postgres` (`prefect-postgres`)	postgres:16	Shared store: Prefect, MLflow metadata, config store, prediction log.
`minio` (+ `minio-init`)	minio/minio	S3-compatible production model storage on-node.
`mlflow-ui` (+ `mlflow-init`)	worker image	Experiment tracking UI and model registry.
`ollama` (+ `ollama-init`)	ollama/ollama	Local LLM (qwen2.5:1.5b-instruct) for `/explain_bid` narratives.
`dashboard` (`smarthub-dashboard`)	dashboard image	Streamlit multipage app (leads / monitoring / config).
`watchtower`	containrrr/watchtower	Polls the registry and recreates updated containers (continuous deploy).

4.4 Data stores

- **DuckDB** — single-file raw store with native SQL upsert and window reads; the pipeline's read/write store.
- **Parquet** — one file per calendar day, merged and de-duped per partition; a lock-free copy dashboards and serving can read without contending with the writer.
- **PostgreSQL** — shared backing store for Prefect state, MLflow metadata, the business-settings config store, and the `smarthub_prediction_log` .
- **S3 / MinIO** — production model artifacts (`.pkl` + manifest + serving pointer) for promoted versions.

4.5 Configuration model (three tiers)

Configuration is split by *who owns it*.

Tier	What	Where	Edited by
Secrets / connection	SSH + Redshift creds, DB URLs, storage paths, passwords	`.env`	ops
Business settings	`target_cm` , `bid_floor` , `bid_max_cap` , `min_source_quality`	Postgres config store, via the Streamlit **Config** page	business
Task configs	model type, training window, calibration, feature selection, bid step, SHAP mode	`config/*.yaml`	developers (git)

Task-config keys fall back to code defaults when absent; business settings are typed, validated, and history-tracked in Postgres.

4.6 Orchestration, CI/CD, and deployment

Each pipeline stage has a **Prefect-free core** plus a thin **Prefect flow** wrapper; deployments (`data-pull → build-features → train-model`) are cron-scheduled per lead type onto a process work-pool served by one worker. On every push/PR, **GitHub Actions** runs isort + black + flake8 + pytest across a Python 3.11/3.12 matrix; only on the deploy branch, after the matrix passes, it builds and pushes the `serve` , `worker` , and `dashboard` images, which **watchtower** on the node auto-deploys. The recommended host for the current target is an AWS `m7i.2xlarge` (8 vCPU / 32 GB) with a gp3 200 GB data volume, co-located with Redshift.

---

5. Low-Level Design (LLD)

This section defines each block: its responsibility, key modules, inputs and outputs, and the important internal mechanics. Figure 2 shows the module-level component architecture and how data and control flow between packages.

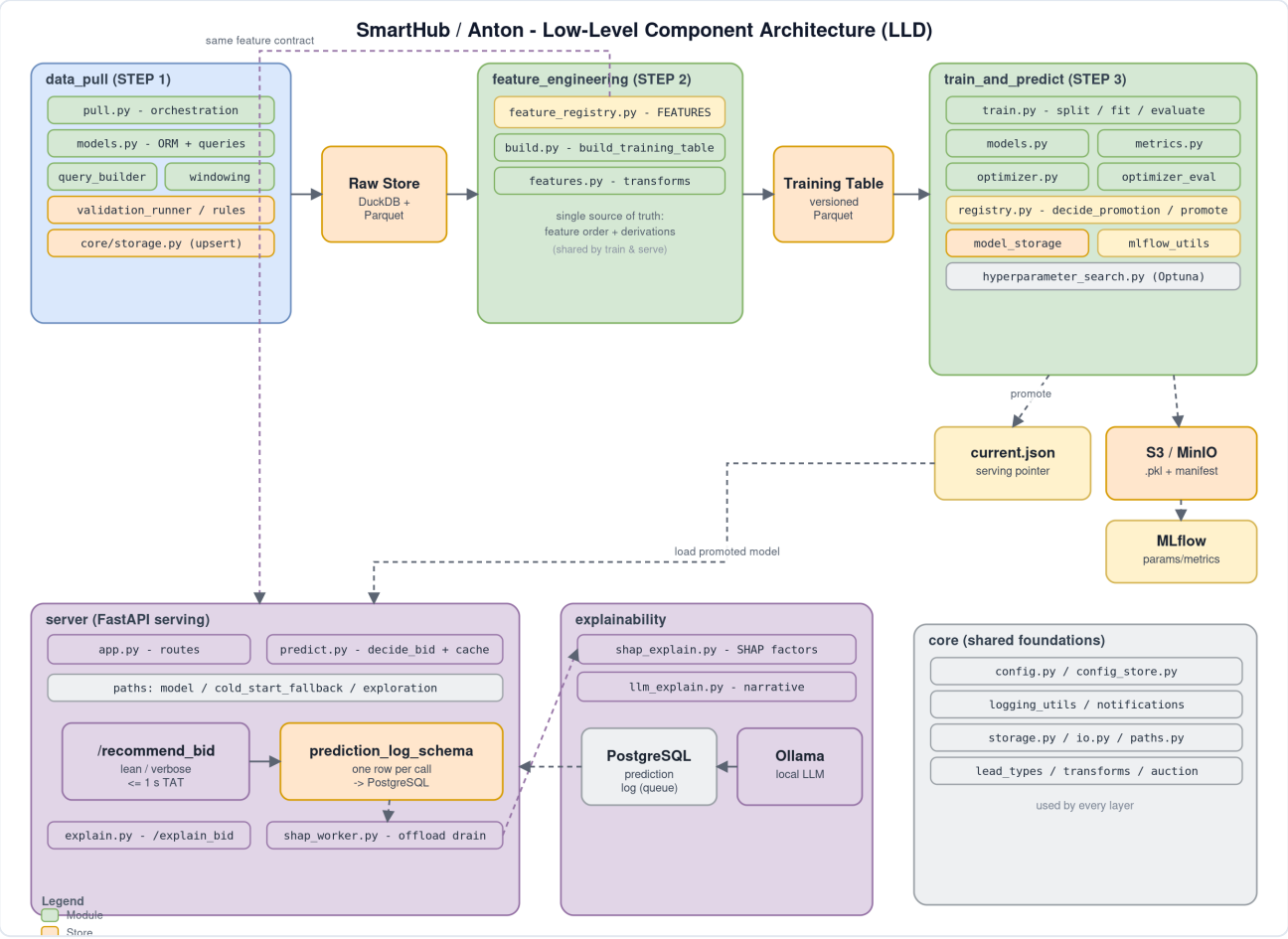

Figure 2 — Low-level component architecture. Boxes are code modules grouped by package (`data_pull` , `feature_engineering` , `train_and_predict` , `server` , `explainability` , `core`); solid arrows are the offline data flow, dashed arrows are serving/async links. The feature registry is the shared feature contract used by both training and serving.

5.1 Data-Pull block

Responsibility. Extract `lead_pings` for a time window and lead type from Redshift into the raw store, incrementally and idempotently.

Key modules. `data_pull/pull.py` (orchestration + connection), `data_pull/models.py` (SQLAlchemy ORM + query builders), `data_pull/query_builder.py` (curated column list from the field registry), `data_pull/windowing.py` (watermark window math), `data_pull/flow.py` (Prefect wrapper), `core/storage.py` (persistence).

Mechanics. `pull.py` opens an `SSHTunnelForwarder` to the bastion and forwards to Redshift (or connects directly when in-VPC), then builds a SQLAlchemy engine on the `redshift+redshift_connector` dialect. Queries are expressed with the ORM expression language (never string-interpolated), producing `SELECT <curated columns> WHERE created_at ∈ [lower, upper) ORDER BY created_at` , optionally LEFT-JOINed to a per-ping expected-revenue subquery. The pull is **incremental with overlapping windows**: a per-lead-type **watermark** (a Prefect Variable holding the latest `created_at` seen) drives each run to re-read from `watermark − overlap_hours` to now, so late-resolving outcomes (`won` , `rev` , listing payouts) update in place. Updates land via **upsert keyed on** `id` , so overlapping windows never duplicate rows. On the very first run (no watermark) a backfill lookback is used.

Persistence (`core/storage.py`). Selected by `STORAGE_BACKEND` : DuckDB (delete-by-id + insert inside a transaction, with schema evolution), Parquet (per -day partition, written to a temp file and atomically renamed), or both. Both backends de-dupe on `id` , preferring the most recent `updated_at` .

Output. Rows in DuckDB and/or dated Parquet partitions; a per-lead-type watermark advanced to the max `created_at` pulled.

5.2 Data-Validation block

Responsibility. Assess the quality of each freshly-pulled batch and report — **detect-and-report only**; it never drops, imputes, caps, or blocks the pull.

Key modules. `data_pull/validation_rules.py` (pandera schema + pandas checks), `data_pull/validation_runner.py` (orchestrates + assembles a `ValidationReport`), `data_pull/validation_custom.py` (per-field custom rules), `data_pull/validation_report.py` (renders to log / Slack / Prefect artifact).

Checks. A **pandera** layer validates dtypes, numeric ranges (e.g. `age` 1–120, `bid ≥ 0`), and categorical domains (state, gender, marital status), sourced from the field registry. A **pandas** layer adds cross-field integrity checks (e.g. `won=true` with no bid; `current_carrier` populated while `insured=false`), a per-column null/blank-rate catalogue, constant-column detection, and batch quality metrics. Errored and auction-ineligible rows are counted separately so they don't distort feature-quality reporting. If pandera is absent, schema checks degrade to a warning and the pandas checks still run.

Output. A structured `ValidationReport` rendered to a per-lead-type Prefect artifact, a "Data quality" section in the Slack notification, and a CLI log summary.

5.3 Feature-Engineering block

Responsibility. Transform the raw store into an immutable, versioned, leakage-safe training table with a stable feature contract.

Key modules. `feature_engineering/feature_registry.py` (the `FEATURES` registry), `feature_engineering/features.py` (transforms + column resolution), `feature_engineering/build.py` (`build_training_table`), `feature_engineering/flow.py` (Prefect wrapper).

Feature registry. A single dict of frozen `FeatureSpec` records is the one source of truth for every feature: its kind (numeric / categorical / binary), raw-vs-derived, which lead types it applies to, whether it's mandatory, and the derivation function for computed features (e.g. `is_workday` from the holiday calendar and `pst_date`; `age_cohort` from `age`; `multi_vehicle` from `num_vehicles`). **Insertion order is the canonical model-feature order**, which keeps the monotone-constraint vector aligned between train and serve time.

Training-table build. `build_training_table` drops errored pings, coalesces revenue and derives the win label, applies the registry derivations, normalises categorical dtypes to strings (missing → `"__MISSING__"` / `"NAvail"`) and numerics (missing `age` → `-1` + `age_missing`), and explicitly excludes known leakage columns. Feature **selection** is configurable per run: each lead type has a locked mandatory core plus an optional set toggled in YAML (`all` / `none` / explicit list); training and serving resolve columns through the *same* registry function so they never drift.

Output. An **immutable, versioned training-table Parquet** (timestamp version + a `.json` lineage manifest recording `data_min/max_created_at` and row count), never overwritten, bounded by the rolling recency window.

5.4 Training block

Responsibility. Fit and evaluate a calibrated, monotone win-probability model for one lead type from a chosen training-table version.

Key modules. `train_and_predict/train.py` (orchestration), `train_and_predict/preprocessing.py` (load + normalise + split helpers), `train_and_predict/models.py` (pipeline builders), `train_and_predict/metrics.py` (evaluation), `train_and_predict/flow.py` (Prefect wrapper).

Pipeline. A scikit-learn `Pipeline` of a `ColumnTransformer` (passthrough or scale numeric; ordinal/one-hot encode categorical) plus a classifier. Three model families are supported — **LightGBM** (default), **XGBoost**, and **logistic regression**. The tree models apply a **monotone constraint of 1 on `bid`** (0 on all other features), with the constraint vector aligned to the registry's feature order. Optional **isotonic calibration** (`CalibratedClassifierCV`, cv=3) keeps probabilities well-scaled while preserving monotonicity.

Split & evaluation. `prepare_training_data` sorts by `created_at`; the split is **chronological** by default (last `test_size` fraction as the test set) or stratified-random. Evaluation reports ROC/PR AUC, log-loss, Brier score, and calibration error, plus train/test feature-coverage diagnostics.

Output. A fitted model object, an evaluation summary, and the inputs to the promotion decision.

5.5 Bid-Optimizer block

Responsibility. Given a fitted model and a lead, choose the expected-profit-maximising bid; and, offline, replay that choice over a holdout to score the model economically.

Key modules. `train_and_predict/optimizer.py` (per-row bid search + chunked scoring), `train_and_predict/optimizer_evaluation.py` (holdout replay + monotone / profit summaries). The same core is used online by `server/predict.py`.

Mechanics. For a lead, generate the candidate-bid grid from `min_bid` up to `expected_revenue × (1 − target_cm)` in `bid_step` increments, score every candidate with a single vectorised `predict_proba`, compute `expected_profit = win_rate × (expected_revenue − bid)` per candidate, and select the maximum. The evaluation runs this over the full holdout in chunks (default 500 rows) and summarises expected-profit lift vs the current bids and a **monotonicity** check (violation rate vs a configured tolerance).

Output. A recommended bid + predicted win-rate/profit per row; aggregate optimiser and monotonicity summaries used by the promotion gate.

5.6 Promotion & Model-Registry block

Responsibility. Decide whether a freshly-trained "challenger" should replace the currently-serving model, and persist/flip versions atomically.

Key modules. `train_and_predict/registry.py` (`decide_promotion` , version assignment, promote), `train_and_predict/model_storage.py` (Filesystem + S3 stores), `train_and_predict/mlflow_utils.py` (MLflow mirror).

Promotion gate. Promotion is gated first on **bid-monotonicity** (the challenger must pass the configured max violation rate), then on a comparison against the currently-serving model evaluated on the *same* holdout via `decide_promotion` : absolute thresholds (`max_log_loss` , `min_expected_profit`) and relative thresholds (`max_log_loss_regression` , `min_profit_ratio` , `max_absolute_profit_loss_tolerance`). Automatic mode promotes on pass; manual mode marks the run "awaiting manual promotion".

Registry & storage. Every run writes a `run_<ts>_<uuid>.pkl` (joblib) plus a `.json` manifest capturing the full resolved config. A promoted run also receives a **collision-free sequential production version** (`auto_v9` , `home_v1`), claimed atomically (`open('x')` on the filesystem / conditional `PutObject` on S3), and updates a `current.json` **serving pointer**. Promotion is **transactional**: publish artifact + manifest + pointer to production storage first, then mark the run promoted and flip the local pointer — so a failed publish never leaves a model falsely "serving." Production-storage failures are surfaced (raise), not silently downgraded to local. Everything is mirrored into MLflow.

Output. A promotion decision + reason; on promotion, a new production version and an updated serving pointer.

5.7 Serving block

Responsibility. Answer `/recommend_bid` on the hot path within the latency SLA, and log every decision with full lineage.

Key modules. `server/app.py` (FastAPI app + routes), `server/predict.py` (`decide_bid` , model cache, request/response schemas, async log writer).

Hot path. On startup each uvicorn worker warm-loads the promoted model (cached per worker, so no disk load sits on a request). Per request it builds one model-ready feature row (via the shared feature transform), runs the bid optimiser, and returns the **max-expected-profit bid**. `decide_bid` always takes exactly one explicit, logged path: **model** (normal), **cold_start_fallback** (no model yet), or **exploration** (a scheduled, reproducible probe that perturbs the optimal bid to keep learning the market). The response is **lean by default**; `verbose: true` returns the full decision payload.

Off-path work. The **prediction log** (one row per call) is written asynchronously to Postgres by a decoupled writer, and **SHAP enrichment** is offloaded (see 5.8) — neither inflates bid latency. A single `nginx` reverse proxy is the only host-exposed entry point.

5.8 Explainability block

Responsibility. Explain an already-logged prediction without touching the bid path.

Key modules. `server/shap_worker.py` (offload worker), `train_and_predict/shap_explain.py` (SHAP over the calibrated pipeline), `train_and_predict/llm_explain.py` + `server/explain.py` (LLM narrative), `server/predict.py` `/explain_bid` route.

SHAP offload. Behind `shap_enrichment_mode` (`inprocess` | `offload` | `off`), the default target is **offload**: serving logs the prediction with SHAP empty, and the separate `shap-worker` drains those rows from the prediction-log table (a DB-as-queue claimed with `FOR UPDATE SKIP LOCKED` for safe concurrency) and backfills the identical explanation. SHAP runs in log-odds space over the LightGBM estimators; the explainer is cached per model version and a single calibration fold is explained for speed. The row for SHAP is reconstructed from the logged `model_input_features` with dtypes normalised to match the encoder.

LLM narrative. `/explain_bid` optionally turns the SHAP factors into a plain-English narrative via a **local Ollama** model — never on the bid path.

5.9 Prediction-Log block

Responsibility. Persist a complete, queryable record of every prediction decision for audit, monitoring, and SHAP backfill.

Key module. `train_and_predict/prediction_log_schema.py` (single-table `smarthub_prediction_log` , schema-versioned; JSON columns stored as TEXT).

Contents. Each row captures the prediction id, timestamps, `model_version` and canonical package `package_version` , the request context, the `model_input_features` , the selected bid and its predicted win-rate/profit, the decision path/reason, per-candidate optimizer evaluations, and (async) the SHAP payload. NaN values are sanitised to null so the JSON stays valid; missing columns are added idempotently under concurrency.

5.10 Monitoring block

Responsibility. Give business and engineering a live view without contending with the pipeline.

Key modules. `monitoring/*` — a single multipage **Streamlit** app: a **Leads** explorer (filters, funnel, win-rate curves), a **Monitoring** page (revenue / CM / win-rate over time), a password-gated **Config** page that reads/writes the business-settings store, plus health/SLO views. Dashboards read the lock-free **Parquet** copy, so they never contend with the pipeline's DuckDB write lock.

5.11 Cross-cutting concerns

- **Logging (`core/logging_utils.py`).** Human-readable text to stdout (clean `docker logs`) plus, when `SMARTHUB_LOG_JSON_DIR` is set, rich rotating JSON logs to file — a dual sink.
- **Notifications (`core/notifications.py`).** Slack messages for data-quality summaries, pull success/failure, and CI status.
- **Config store (`core/config_store.py`).** Typed, validated, history-tracked business settings in Postgres, edited via the Streamlit Config page.
- **Package version (`smarthub.__version__`).** A canonical semver recorded on every prediction alongside `model_version` for full reproducibility.

---

6. API Reference

The serving API exposes the bid-recommendation model over HTTP behind nginx. `application/json` for all `POST` requests; auth is network-restricted at the edge today. Every prediction is logged server-side to `smarthub_prediction_log` with a unique `prediction_id` , returned in the response. A standalone, fully formatted API reference is also published as `docs/API.pdf` .

Method	Path	Purpose
`POST`	`/recommend_bid`	Return the expected-profit-maximising bid for one lead.
`POST`	`/explain_bid`	Explain an already-logged prediction (SHAP + optional LLM narrative).
`GET`	`/health`	Liveness + which model is loaded.

6.1 `POST /recommend_bid`

Runs one lead through the bidding policy (`decide_bid`) and returns the recommended bid. The response is **lean by default**; pass `verbose: true` to also receive the full decision payload used by the logging schema.

Request body (`BidRequest`) — key fields.

Field	Type	Required	Default	Notes
`expected_revenue`	number	Yes	—	Revenue if won (> 0); drives the profit calc.
`target_cm`	number	no	`0.25`	Target contribution margin; sets the bid ceiling (`0 ≤ x < 1`).
`min_bid`	number	no	`0.25`	Partner-side floor / lowest candidate bid (`≥ 0`).
`bid_step`	number	no	`0.25`	Candidate-bid increment (`> 0`).
`campaign_id`	integer	Yes	—	Campaign the lead belongs to.
`lead_type_id`	integer	no	`6`	Which model to use (`6` = auto, `1` = home).
`created_hour`	integer	Yes	—	Pacific hour of day (`0–23`).
`created_dayofweek`	integer	Yes	—	Day of week (`0` = Mon … `6` = Sun).
(optional lead features)	mixed	no	`null`	`state` , `insured` , `home_owner` , `age` , `num_vehicles` , etc.; missing values are imputed by the model pipeline.
`verbose`	boolean	no	`false`	When `true` , return the full decision payload. Never affects the bid.

Only `expected_revenue` , `campaign_id` , `created_hour` , and `created_dayofweek` are strictly required; any unlisted field is ignored.

Response (lean) — key fields.

Field	Type	Meaning
`recommended_bid`	number | null	The chosen bid. `null` = no viable bid (revenue too low to bid at/above the floor while meeting `target_cm`).
`recommended_bid_predicted_win_rate`	number | null	Model-predicted win probability at the chosen bid.
`recommended_bid_predicted_profit`	number | null	Predicted profit at the chosen bid (`win_rate × (expected_revenue − bid)`).
`max_bid`	number	The ceiling (`expected_revenue × (1 − target_cm)`).
`n_candidate_bids`	integer	Candidates evaluated in the sweep.
`decision_path`	string	`model` , `cold_start_fallback` , or `exploration` .
`decision_reason`	string	Human-readable explanation of the decision.
`model_version`	string	The serving model version that produced the bid.
`prediction_id`	string (uuid)	Log key; use it with `/explain_bid` .

`verbose: true` additionally returns the full model-input feature row and a capped candidate-bid sweep (selected bid + first candidates), i.e. the payload persisted to the prediction log.

Example.

```
curl -s -X POST http://localhost:8000/recommend_bid \
  -H 'Content-Type: application/json' \
  -d '{
        "expected_revenue": 100,
        "campaign_id": 13,
        "created_hour": 10,
        "created_dayofweek": 2,
        "state": "CA",
        "age": 34,
        "num_auto_accidents": 1
      }'
```

6.2 `POST /explain_bid`

Explains an already-logged prediction. Takes a `prediction_id` (and optional flags to include the LLM narrative), returns the SHAP factor breakdown (top positive/negative contributors to the win probability) and, optionally, a plain-English narrative generated by the local LLM. This route is never on the bid path; if SHAP for that row has not yet been backfilled it returns a "pending"/unavailable status rather than blocking.

6.3 `GET /health`

Returns service liveness and the resolved model status for a lead type (`/health?lead_type_id=6`): whether a model is loaded, its version, and cold-start status.

6.4 Errors

Validation errors (missing required field, out-of-range value) return HTTP `422` with a field-level detail body (FastAPI/pydantic). A `recommended_bid` of `null` is **not** an error — it is a valid "no viable bid" decision within a `200` response.

---

7. Known Limitations & Roadmap

This section draws on the internal data-pull and training audits and the open product items. Items marked **(fixed)** have already been remediated.

7.1 Data pipeline

- **(fixed)** Scheduled data-pull flow kwarg mismatch that raised `TypeError` every run; DuckDB upsert made transactional (no data loss on mid-upsert failure); Parquet partition writes made atomic (temp file + rename).

- **Validation is warn-only.** Even an empty, schema-drifted, or all-null-critical pull is persisted and the watermark advances. A circuit breaker on catastrophic conditions (zero rows, missing columns, volume collapse) is recommended.
- **Watermark can skip late-arriving rows.** Anchoring on `created_at` with a small fixed overlap means rows inserted late but stamped older than `watermark − overlap` are never re-pulled. Consider an ingestion/ `updated_at` watermark or a periodic wide reconciliation pull.
- **Whole-result-in-memory / no query timeout.** Large windows load fully into memory; only a connection (not statement) timeout is set. Cap/stream large windows and add a statement timeout.

7.2 Training

- **Config loads at import time** — a missing/invalid YAML hard-fails unrelated imports and freezes env overrides; make it lazy.
- **HPO determinism / leakage** — the `random` split-reservation relies on stable row order, and Optuna with `n_jobs > 1` is non-reproducible. Partition on a stable key; pin `n_jobs` when reproducibility matters. (The default `time` split is safe.)
- **Edge-case robustness** — `evaluate_model` raises on a single-class test set; an evaluation-summary `json.dumps` lacks `default=str` (fails on NumPy scalars); MLflow "already registered?" swallows transient errors (risking duplicate registration) and a silent alias-set failure can leave production without a pointer.

7.3 Product roadmap

- **Exploration around the optimum** — evolve from fixed `dumb 10/25/50/75 ±10%` probing to deliberate explore/exploit around Anton's own optimum.
- **Feedback loop** — close the loop so the model converges automatically on the live win-rate curve (the current Anton did not).
- **Expected-revenue source** — complete the switch to backend `exp_rev` once it is fully populated across lead types, retiring the interim listings-sum.
- **Backtest / A-B vs current Anton** — the new model must be **≥** the current system on profit/CM; validate before placing live bids.
- **Concurrency hardening** — a per-partition lock for Parquet writes to prevent lost updates under simultaneous writers (atomicity is already handled).

Appendix A. Glossary

Term	Meaning
Ping	A lead offer from a partner ("what will you pay?").
Partner / account	Upstream producer that sells leads to SmartHub.
Buyer	Downstream agent SmartHub resells to.
Campaign	A partner's lead stream (`campaign_id`).
Won	The partner accepted our bid; SmartHub holds the lead.
Bid	What SmartHub offers the partner for the lead (our cost).
Expected revenue	Reject-discounted forecast revenue; the ceiling input.
Contribution margin (CM)	Profit ÷ revenue; also the bidding-strategy lever.
Win rate	Share of bids that win the lead, across price points.
Ceiling (max bid)	`expected_revenue × (1 − target_cm)` .
Floor (min bid)	A partner-imposed minimum bid; may not exist.
Shelf / edge	A price point where win rate barely moves — bid the cheaper side.
Exploration	Deliberately bidding around the optimum to learn the market.
Recency window	The rolling lookback that defines "recent" data.
Cold start	No recent data (new partner/lead type) → explicit fallback.
Watermark	Per-lead-type marker of the latest `created_at` pulled.
Serving pointer	`current.json` naming the promoted model in use.

Appendix B. Repository layout

```
src/smarthub/
  core/               shared foundations: config, config_store, paths, logging,
                      notifications, storage, io, transforms, lead_types
  data_pull/          STEP 1: pull, query_builder, windowing, field_registry,
                      validation_*, flow (Prefect), cli
  feature_engineering/STEP 2: features, feature_registry, build, flow
  train_and_predict/  STEP 3: train, models, registry, model_storage, optimizer,
                      metrics, mlflow_utils, hyperparameter_search, shap_explain,
                      llm_explain, prediction_log_schema, flow
  server/             FastAPI bid API: predict, app, explain, shap_worker
  monitoring/         Streamlit multipage app (leads / monitoring / config)
config/               smarthub.yaml, training.yaml, holidays.json
docker/               Dockerfile.app / .worker / .serve, nginx/
docs/                 CONTEXT, MODELING, PROJECT_OVERVIEW, API, audits, this doc
.github/workflows/    ci_cd.yml (lint + test + build/push)
docker-compose.yaml   the single-node stack
```

Appendix C. Technology stack

Area	Tools
Language / runtime	Python 3.10+ (CI 3.11 / 3.12)
Data wrangling	pandas, numpy, pyarrow
Stores	DuckDB, Parquet, PostgreSQL, MinIO (S3)
Source DB / connectivity	Redshift, redshift-connector, SQLAlchemy, sshtunnel
Validation	pandera + pandas checks (warn-only)
ML training	scikit-learn, LightGBM, XGBoost; CalibratedClassifierCV (isotonic)
Hyperparameter search	Optuna (TPE)
Experiment tracking	MLflow
Model persistence	joblib, boto3 (S3/MinIO), filesystem
Explainability	SHAP, Ollama (local LLM)
Serving	FastAPI, uvicorn, pydantic, nginx
Dashboards	Streamlit, plotly
Orchestration	Prefect 3
CI/CD & deploy	GitHub Actions, Docker Hub, watchtower
Code quality	black, isort, flake8, pytest

End of document.